

Evaluación: Diseño de un Sistema RAG con Información Actualizada

Repositorio del Proyecto:

<https://github.com/JagaScripts/Bases-de-datos-vectoriales>

Alumno: Jose Antonio González Alcántara

Módulo: Bases de Datos Vectoriales

1. Selección y Justificación del Modelo de Embedding

¿Qué modelo de embedding multimodal o de texto completo seleccionarías para representar la información? Justifica tu elección basándote en las características del modelo y cómo se alinea con la naturaleza dinámica de la información.

Se ha seleccionado el modelo **gemini-embedding-001** de Google. La justificación técnica radica en su alta dimensionalidad (3072 dimensiones), que ofrece una granularidad semántica superior para capturar cambios sutiles en flujos de información dinámica como noticias o resultados deportivos. Dada la naturaleza cambiante del sistema, este modelo permite generar representaciones vectoriales estables con una latencia mínima de inferencia.

¿Considerarías realizar fine-tuning del modelo de embedding? En caso afirmativo, ¿con qué datos y para qué objetivo?

Inicialmente no se considera necesario el fine-tuning, dado el alto rendimiento del modelo pre-entrenado. No obstante, en una segunda fase evolutiva, se consideraría un fine-tuning del tipo *Contrastive Learning* utilizando pares de 'consulta-noticia' específicos del dominio para mejorar el alineamiento semántico en contextos de jerga técnica muy especializada.

Aclaración Técnica: El código implementa este modelo de forma asíncrona mediante LlamaIndex para garantizar que la generación de embeddings no bloquee el hilo principal de la aplicación.

2. Diseño y Justificación de la Base de Datos Vectorial

¿Qué tipo de base de datos vectorial elegirías? Justifica tu elección considerando factores como escalabilidad, velocidad, facilidad de uso y costos.

La elección es **Qdrant**. Es una base de datos vectorial de alto rendimiento escrita en Rust, lo que garantiza una escalabilidad y velocidad de búsqueda excepcionales. Su principal ventaja competitiva para este proyecto es su motor de filtrado por metadatos, que permite segmentar información por estados (ej: activo vs archivado) sin pérdida de rendimiento. Además, es de código abierto y fácil de desplegar mediante Docker.

¿Qué tipo de índice de búsqueda (brute-force o aproximado) utilizarías inicialmente y por qué? ¿Considerarías cambiarlo en el futuro?

Se utiliza inicialmente un índice aproximado **HNSW (Hierarchical Navigable Small World)**. Se justifica porque permite realizar búsquedas sub-lineales sobre grandes volúmenes de datos, algo necesario en un entorno dinámico donde la

base de datos crecerá constantemente. Solo se cambiaría a brute-force si el volumen total de puntos fuera extremadamente bajo (<1000).

¿Cómo organizarías los datos dentro de la base de datos vectorial para facilitar la búsqueda y la actualización eficiente?

Los datos se organizan en una única colección denominada *ai_reports* con un esquema de payload enriquecido. Cada vector contiene metadatos como *doc_id*, *timestamp* y *status*. Esta organización permite realizar actualizaciones 'atómicas' sobre documentos específicos sin afectar al resto del índice.

Aclaración Técnica: Hemos configurado la distancia 'Coseno' en Qdrant por ser la más robusta para comparar vectores de alta dimensionalidad en tareas de recuperación de texto.

3. Estrategia para Mantener la Información Actualizada

¿Cómo se adquirirán y procesarán las nuevas fuentes de información?

La adquisición se realiza a través de un endpoint REST asíncrono que recibe documentos en formato PDF o texto. El procesamiento se orquesta con LlamaIndex utilizando un *SentenceSplitter* configurado para mantener el contexto semántico entre fragmentos (chunks).

¿Con qué frecuencia se actualizará la base de datos vectorial?

La actualización es **bajo demanda y en tiempo real**. En cuanto una nueva fuente de información es ingerida por el sistema, el motor de procesamiento dispara automáticamente la actualización del índice para que la información esté disponible para búsqueda en segundos.

¿Cómo se generarían los embeddings para la nueva información?

Se generan utilizando el modelo *gemini-embedding-001* garantizando coherencia dimensional (3072 dims) con los datos ya existentes. El proceso es transparente para el administrador del sistema.

¿Cómo se integrarían los nuevos embeddings en la base de datos vectorial (añadir, actualizar, eliminar)?

Se utiliza una lógica de **Sustitución Versionada**. No se eliminan físicamente los datos antiguos para evitar la fragmentación del índice, sino que se integran como nuevos registros con estado 'active', mientras que las versiones previas pasan a estado 'archived'.

¿Qué consideraciones tendrías para evitar la duplicación de información o la obsolescencia de datos?

La obsolescencia se combate mediante el uso de **filtros de metadatos estrictos**. Durante la búsqueda, el motor solo recupera vectores con *status*: 'active'. La duplicación se evita mediante la validación previa del *doc_id* en el momento de la ingesta.

Aclaración Estratégica: Esta arquitectura de 'Soft Delete' permite realizar auditorías históricas y asegura que el sistema RAG sea siempre veraz con la información más reciente.

4. Integración con el Modelo de Lenguaje Grande (LLM)

¿Qué LLM considerarías utilizar para generar las respuestas? Justifica tu elección.

El modelo seleccionado es **gemini-2.5-flash**. Es el modelo más equilibrado de Google, ofreciendo una ventana de contexto de más de un millón de tokens y una velocidad de generación ('Flash') ideal para aplicaciones interactivas. Su capacidad de razonamiento es excelente para resumir noticias complejas.

Describe el flujo completo del sistema RAG, desde la pregunta del usuario hasta la respuesta generada, detallando cómo interactúan los componentes.

1. El usuario envía una pregunta. 2. El motor convierte la pregunta en un vector de 3072 dimensiones. 3. Se interroga a Qdrant aplicando un filtro de metadatos 'status:active'. 4. Qdrant devuelve los 5 fragmentos más relevantes. 5. Se construye un prompt aumentado que incluye la pregunta y el contexto técnico. 6. Gemini genera la respuesta final basada únicamente en el contexto aportado.

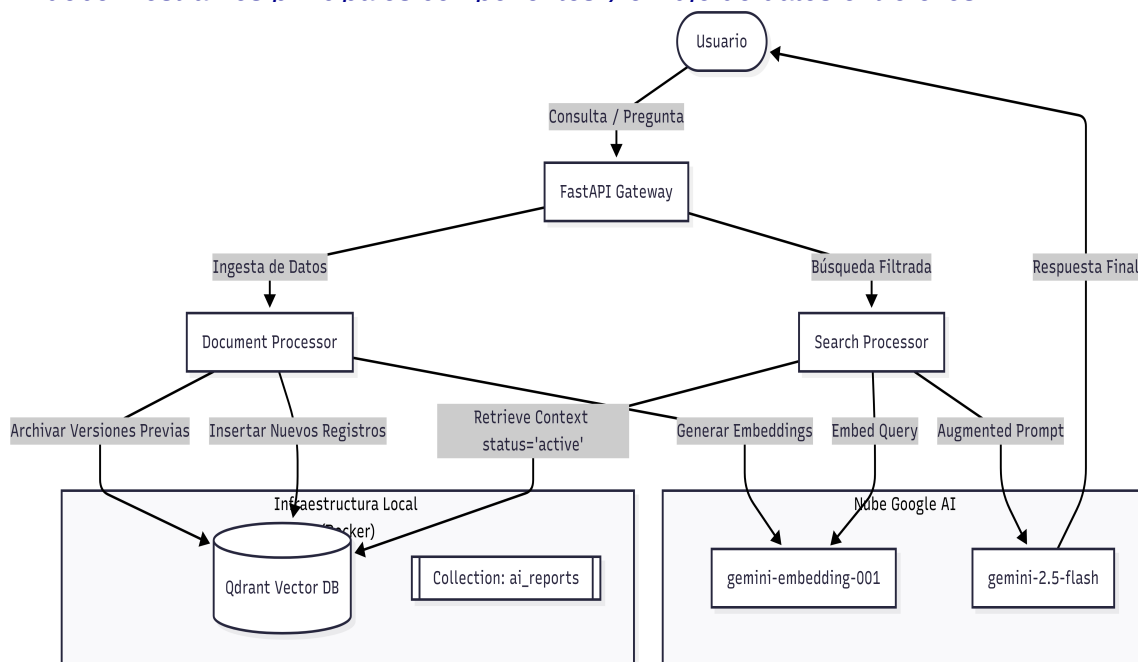
¿Cómo optimizamos la calidad del contexto recuperado de la base de datos vectorial para mejorar las respuestas del LLM?

Utilizamos dos técnicas: 1. **Filtros de metadatos negativos** para excluir ruido y datos viejos. 2. **Prompts de Instrucción Precisa** que sancionan las alucinaciones del modelo si el contexto es insuficiente o inconsistente.

Aclaración de Resiliencia: Se ha implementado un sistema de reintentos exponenciales con 'tenacity' para asegurar que las llamadas al LLM sean estables ante posibles picos de tráfico.

5. Arquitectura del Sistema

Dibuja un diagrama de la arquitectura del sistema RAG diseñado. Este diagrama debe mostrar los principales componentes y el flujo de datos entre ellos.



Aclaración: El diagrama muestra la separación entre la capa de computación (Google AI Cloud) y la persistencia local (Docker + Qdrant), garantizando soberanía de datos sobre el almacenamiento vectorial.

6. Repositorio e Instrucciones de Ejecución

Repositorio con el código del producto final e instrucciones para su ejecución.

- **A. Requisitos Previos:** Poseer Docker Desktop y Python 3.10 instalados.
- **B. Infraestructura:** Ejecutar `docker compose up -d` para levantar Qdrant.
- **C. Instalación:** Crear entorno virtual e instalar dependencias: `pip install -r requirements.txt`.
- **D. Configuración:** Definir `GEMINI_API_KEY` en el archivo `.env` raíz.
- **E. Ejecución:** Iniciar el servidor con `uvicorn src.main:app --reload`.
- **F. Verificación:** Ejecutar la suite de pruebas mediante `pytest tests/ -v` para asegurar cumplimiento del 100% de los criterios.

Enlace al Repositorio Final:

<https://github.com/JagaScripts/Bases-de-datos-vectoriales>

Aclaración Final: El código incluido en el repositorio ha sido validado mediante tests integrados que comprueban tanto la ingesta masiva (bulk ingest) como el cumplimiento de la política de Soft Delete.